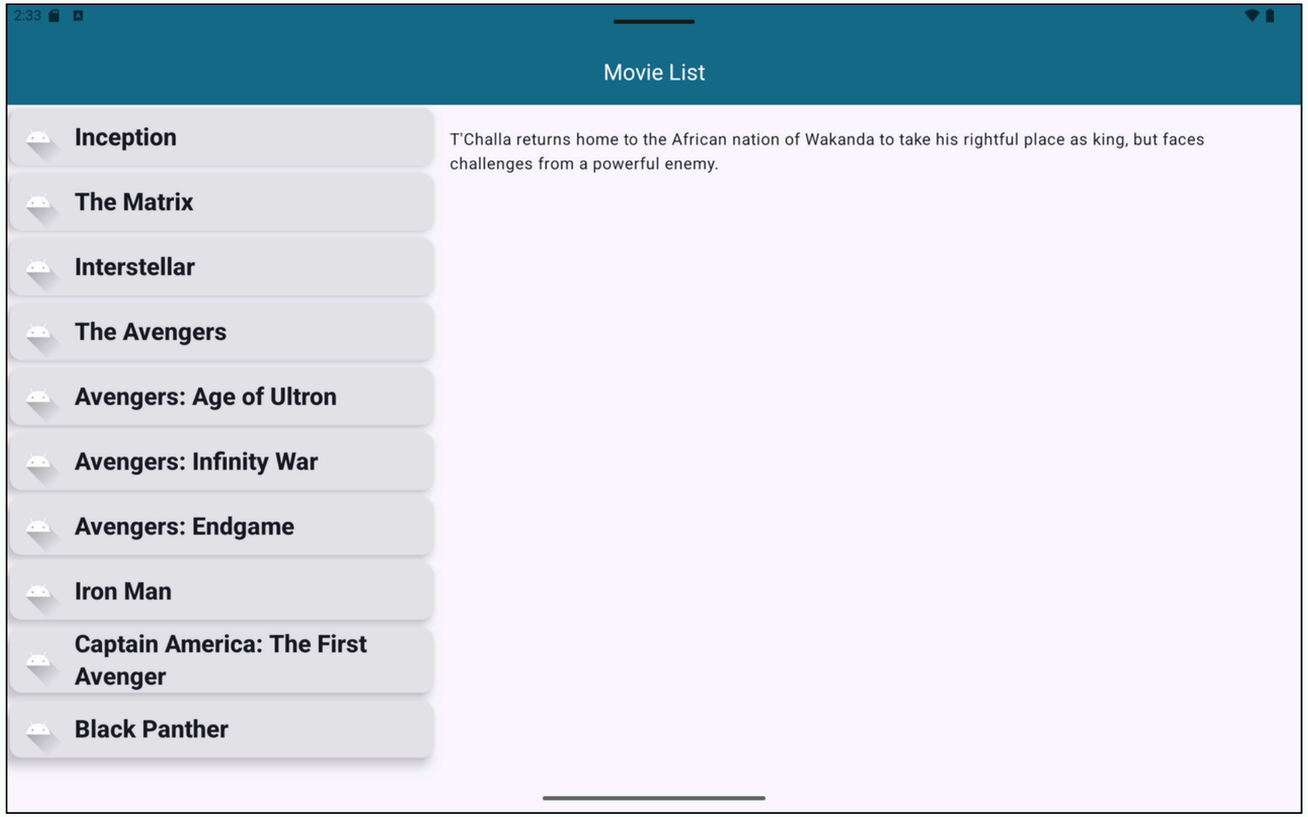
Kotlin

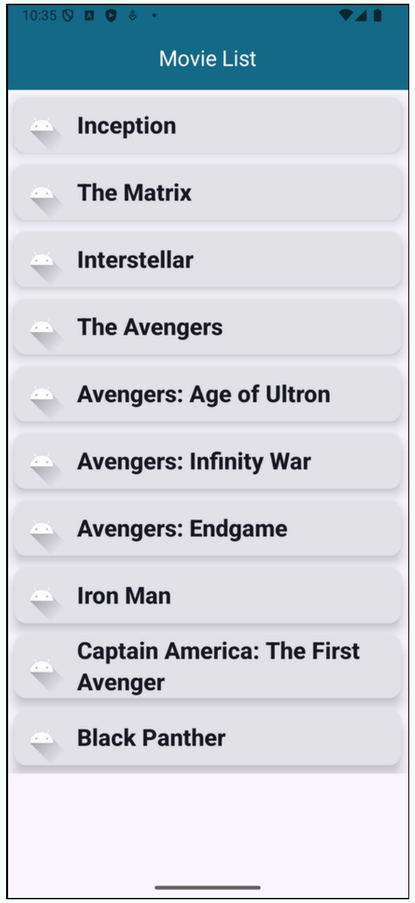
MOBILE PROGRAMMING

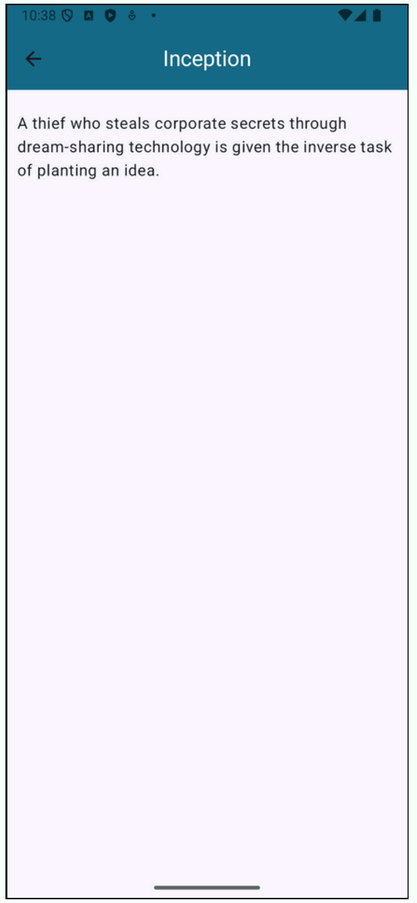
KOTLIN RESPONSIVE LAYOUT

ROBBY TAN

Today’s Topic







Today’s Topic

Android View System (Legacy)

- Using three different layout (.xml file): phone_page.xml, table_page.xml, and detail_page.xml
- Each xml is loaded using Fragment.

Jetpack Compose

- Using three Composable function (PhonePage, TabletPage, DetailPage).
- Use WindowsWidthSizeClass to decide if app is on phone or tablet.

Dependencies Update

build.gradle.kts (Module:app)

```
plugins {
  alias(libs.plugins.android.application)
  alias(libs.plugins.kotlin.android)
  alias(libs.plugins.kotlin.compose)
  alias(libs.plugins.kotlin.serialization)
}
```

libs.versions.toml

```
[versions]
kotlin = "2.2.21"
navigationCompose = "2.9.6"
kotlinxSerializationJson = "1.9.0"
material3WindowSizeClass = "1.4.0"
retrofit = "3.0.0"
loggingInterceptor = "4.12.0"
retrofit2KotlinxSerializationConverter = "1.0.0"
materialIconsExtended = "1.7.8"

[libraries]
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref = "navigationCompose" }
kotlinx-serialization-json = { group = "org.jetbrains.kotlinx", name = "kotlinx-serialization-json", version.ref = "kotlinxSerializationJson" }
androidx-compose-material3-window-size-class1 = { group = "androidx.compose.material3", name = "material3-window-size-class", version.ref = "material3WindowSizeClass" }
retrofit = { group = "com.squareup.retrofit2", name = "retrofit", version.ref = "retrofit" }
logging-interceptor = { group = "com.squareup.okhttp3", name = "logging-interceptor", version.ref = "loggingInterceptor" }
retrofit2-kotlinx-serialization-converter = { group = "com.jakewharton.retrofit", name = "retrofit2-kotlinx-serialization-
converter", version.ref = "retrofit2KotlinxSerializationConverter" }
androidx-compose-material-icons-extended = { group = "androidx.compose.material", name = "material-icons-extended", version.ref = "materialIconsExtended" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref = "kotlinxSerializationJson" }
```

API Service

Main URL: http://fittest.itmaranatha.org/service_json/
Service:

- movie_data.json (**GET**)
- m1.json to m5.json (**GET**)

```
1.  interface ApiService {
2.
3.    @GET("movie_data.json")
4.    suspend fun getAllMovies(): List<Movie>
5.
6.    @GET("m{id}.json")
7.    suspend fun getMovie(@Path("id") id: Int): Movie
8.  }
```

MovieViewModel

- Penambahan fungsi **loadMovie** yang menerima sebuah parameter bertipe Int.
- Fungsi **loadMovie** **tidak diload di awal** karena baru akan dijalankan setelah ada *request* untuk akses API sebuah film.

```
1.  import androidx.lifecycle.ViewModel
2.  import androidx.lifecycle.viewModelScope
3.  import com.robby.kotlin08responsivelayout.entity.Movie
4.  import com.robby.kotlin08responsivelayout.service.ApiClient
5.  import kotlinx.coroutines.flow.MutableStateFlow
6.  import kotlinx.coroutines.flow.asStateFlow
7.  import kotlinx.coroutines.launch
8.
9.  class MovieViewModel : ViewModel() {
10.
11.   private val _movies = MutableStateFlow<List<Movie>>(emptyList())
12.   private val _movie = MutableStateFlow<Movie?>(null)
13.   val movies = _movies.asStateFlow()
14.   val movie = _movie.asStateFlow()
15.
16.   init {
17.     loadMovies()
18.   }
19.
20.   fun loadMovies() {
21.     viewModelScope.launch {
22.       val response = ApiClient.instance.getAllMovies()
23.       _movies.value = response
24.     }
25.   }
26.
27.   fun loadMovie(id: Int) {
28.     viewModelScope.launch {
29.       val response = ApiClient.instance.getMovie(id)
30.       _movie.value = response
31.     }
32.   }
33. }
```

Update AppNavigation (SmartPhone)

- Fungsi ini hanya digunakan oleh SmartPhone.
- Penambahan pengambilan data dan load data tersebut menggunakan ViewModel.

```kotlin
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.robby.kotlin08responsivelayout.ui.phone.DetailScreen
import com.robby.kotlin08responsivelayout.ui.phone.PhoneScreen
import com.robby.kotlin08responsivelayout.viewmodel.MovieViewModel

@Composable
fun AppNavigation(viewModel: MovieViewModel) {
  val navController = rememberNavController()

  NavHost(navController, "home") {
    composable("home") {
      PhoneScreen(
        items = viewModel.movies.collectAsState().value,
        onItemClick = { id -> navController.navigate("detail/$id") }
      )
    }
    composable("detail/{id}") { backStack ->
      val id = backStack.arguments?.getString("id")?.toIntOrNull() ?: 0
      viewModel.loadMovie(id)
      val movie = viewModel.movie.collectAsState().value
      DetailScreen(movie, navController)
    }
  }
}
```

Tablet Page (No Navigation and Backstack)

- Isi Composable TabletContent
  - Row
    - LazyColumn
    - DetailPage

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TableScreen(movieViewModel: MovieViewModel) {
  val movies by movieViewModel.movies.collectAsState()
  Scaffold(
    topBar = {
      CenterAlignedTopAppBar(
        title = { Text("Movie List", color = Color.White) },
        colors = TopAppBarDefaults.topAppBarColors(Color(0xFF176B87))
      )
    }
  ) { innerPadding ->
    TabletContent(movies, modifier = Modifier.padding(innerPadding))
  }
}
```

WindowWidthSizeClass Observer

- Mendapatkan data apakah aplikasi dibuka dengan menggunakan smartphone atau tablet.
- Tidak lagi menggunakan orientation vertical atau horizontal.
- Dapat ditambahkan dengan pemeriksaan **dp** (density pixel).

```
1.  import androidx.compose.material3.windowsizeclass.WindowSizeClass
2.  import androidx.compose.material3.windowsizeclass.WindowWidthSizeClass
3.  import androidx.compose.runtime.Composable
4.  import androidx.lifecycle.viewmodel.compose.viewModel
5.  import com.robby.kotlin08responsivelayout.ui.navigation.AppNavigation
6.  import com.robby.kotlin08responsivelayout.ui.tablet.TableScreen
7.  import com.robby.kotlin08responsivelayout.viewmodel.MovieViewModel
8.
9.  @Composable
10. fun App(windowSizeClass: WindowSizeClass) {
11.   val viewModel: MovieViewModel = viewModel()
12.   val width = windowSizeClass.widthSizeClass
13.
14.   when (width) {
15.     WindowWidthSizeClass.Compact -> {
16.       AppNavigation(viewModel)
17.     }
18.     WindowWidthSizeClass.Medium,
19.     WindowWidthSizeClass.Expanded -> {
20.       TableScreen(viewModel)
21.     }
22.   }
23. }
```



Update MainActivity

```
1.  class MainActivity : ComponentActivity() {
2.    @OptIn(ExperimentalMaterial3WindowSizeClassApi::class)
3.    override fun onCreate(savedInstanceState: Bundle?) {
4.      super.onCreate(savedInstanceState)
5.      enableEdgeToEdge()
6.      setContent {
7.        Kotlin08ResponsiveLayoutTheme {
8.          val windowSizeClass = calculateWindowSizeClass(this)
9.          App(windowSizeClass)
10.       }
11.     }
12.   }
13. }
```

Thank You

ROBBY.TAN@IT.MARANATHA.EDU